SEMTM0010 Transport and Mobility Modelling

Lab sheet: Data-driven modelling

Notes on answers to exercises

Nikolai Bode

1. Direct demand modelling

1.0. How many parameters does this model have? (we didn't make this explicit in the code above)
The IDW model has one key parameter, the exponent that controls the weighting of counts depending on their distance. In the example, the exponent is set to be equal to 2.
1.1. Use inverse distance weighting to predict counts on the entire street network, and see how they change over time (you could create a video showing this).
An animation illustrating a solution has been uploaded onto Blackboard. The worked example in the lab sheet shows how the model can be used for predictions on one node that also includes a sensor. You need to adapt this code, so that you can make predictions for each node in the network we constructed in the worked example, regardless of whether they are associated with a sensor or not.
1.2. Discuss how prediction accuracy could be assessed more rigorously.
In the worked example, we only use a plot for one sensor to qualitatively assess the accuracy of our predictions. A more rigorous assessment would require quantification of accuracy (e.g. mean squared error, or similar). It would also require assessment of prediction accuracy across all sensors, ideally using a leave-one-out or cross-validation scheme (or similar) when training and testing the model where one or more sensors are removed from the data to train the model, and subsequently used to assess accuracy.
1.3. Extend the inverse distance weighting model. You should consider introducing additional variables that can be used in the calculation of distances between nodes (e.g., you could create artificial data on the width of pavements and include this in the distance calculation). You should also consider introducing free parameters that allow you to tune the contribution of sensors to the interpolation (discuss the implication of such an approach).
This is an open-ended exercise. The main point is that you think about how to extend the model by changing the way the weighting for counts is calculated. Instead of using a simple formula based on the distance between locations on the street network, think about how a model for the distance could be developed. A very simple example could be: $$distance_{new} = \left(distance/pavement_width^{\beta}\right)^{\alpha}$$ Where α and β are parameters that can be estimated based on the data. Many other extensions/formulations for this are possible. Since I haven't provided you with pavement widths, you could just create artificial data, e.g. by sampling widths from a uniform distribution between 1m and 5m, or similar.

2. Time series modelling

2.0. play around with the window sizes used to define the trend/seasonal components and discuss how these patterns could arise in a transport context, and what additional patterns could be observed for the same or other transport modes, depending on the time of year, location, or other context.
I am not looking for a particular answer here, this exercise asks you to familiarize yourself with the working of the code that's been provided. Traffic time series often contain daily trends (window size 24 hours) weekly trends (think weekend versus weekdays), and annual trends (think major vacations, climatic conditions, etc.). Changing the window size for the trend will help identify longer-term trends (larger window size), such as general increases in traffic over the course of a year, and vice-versa.
2.1. can you make a case for choosing different time intervals for the seasonality? Try this out in the code.
Yes. See answer to exercise 2.0. Another way of making the case is to try different values and check if the model fit improves (e.g. based on RMSE).
2.2. change the structure of the ARIMA models to explore how this affects model forecasts behaviour (e.g. by including higher-order auto-regressive terms, or similar).
Again, no specific answer is expected here, the idea is to familiarize yourself with how the code and models work. For example, higher-order auto-regressive terms mean that the model 'look further into the past'. In practice, deciding on these values depends on the data, and any knowledge we may have about how the system works.
2.3. implement closed loop forecasting for LSTM example.
For this, you need to make a prediction based on the initial data. Then, rather than feeding in more data, you need to append your prediction to the initial data, and use this to make the next prediction, and so on. In terms of the code, you'll need a for-loop that iteratively makes predictions, and uses these predictions for subsequent predictions.
2.4. try adding additional extraneous predictors to the models (e.g. information on when bank holidays are, or weather data - as I haven't given you this data, simulate it if you would like to try out some code).
Let's illustrate this for the LSTM model and assumed rainfall data. To simulate rainfall data, you could create another time series that is the same length as you traffic time series. You could assign random values from a uniform distribution to this time series. This is obviously nonsensical considering the physics of weather/rainfall and it will add no information to your model but it suffices for testing out some code. You would then have to change the structure of your LSTM model, to allow for the additional feature (rainfall) to be used. Specifically, you would have to change the 'sequenceInputLayer' and then make sure that you follow through with change the remaining code to account for this change.
2.5. discuss/reflect on more robust ways of comparing the different models that go beyond calculating RMSE values for a fixed test set.
Cross-validation, change test and training sets, consider different measures for accuracy…
2.6. use a time series model of your choice to compare the time series at different locations (recall that we have focussed on one location above, for convenience).

What wanted you to do for this exercise is simply fit one of the models provided in the worked example separately to data from different locations (sensors). Then, rather than comparing the raw data between the two locations directly, you could look at the estimated parameter values to see how they differ.
Alternatively, you could use the models fitted in the worked example to make predictions about another location to assess if the predictions are any good, or if they are consistently wrong in some way.
The point here is to explore how models may generalize across locations, how model fit depends on location, and how models can be understood as a simplified summary of time series.
Leading on from this exercise, you could think about how differences between locations could be modelled (e.g. is adding a constant enough to account for differences in traffic volume at different locations?).

2.7. implement an alternative time series model, different to the ones discuss here.

Consult the lecture notes or the internet for alternatives.

3. Machine-learning a microscopic model

3.0. this is probably not the best way of constructing test and training data. Why?

We are sampling randomly from the data to obtain test and training data. This is not systematic, and will make it difficult to compare models.

3.1. discuss how the confusion matrix can be interpreted.

The model in the worked example makes predictions on a 7x7 matrix. In other words, it is a classifier for 49 classes. Not all classes (cells in the 7x7 matrix) occur in the data used, that's why the confusion matrix doesn't show 49 classes.
In the confusion matrix, anything on the leading diagonal (top-left to bottom right) is great, as these are correctly predicted classes (positions). Off-diagonal elements are incorrectly predicted classes (positions). Something that isn't very clear from this confusion matrix is how bad a misclassification is. For example, if the prediction is out by one cell on the 7x7 matrix, that's not too bad but this is not directly captured in the confusion matrix.

3.2. list limitations/poor design choices of the approach shown above.

There are many from the discretization of movement (and the choices we made when doing so, including cell sizes) to the fact that walls and the exit position are not included in the model (as it stands the model assumed that pedestrian movement is only based on the position of other pedestrians). Based on the discussion around exercise 3.1. above, an important issue with the classification-based approach is that we cannot distinguish (and reward the model) for near-misses in position predictions.

3.3. try to improve the model.

This is a very open-ended exercise but one of the obvious things that stands out is that we do not consider the exit location at all in our model. Including this, e.g. by extending the input indication the shortest route to the exit may help improve the model.
State-of-the-art models in this space use substantially more complex Neural Network structures, or they fuse mechanistic models with data-driven models, e.g. by machine-learning the force terms in the social force model. The details of this go beyond the scope of the course. Talk to me if you'd like pointers for relevant references though.